render_md_to_pdf.sh and check_pdf_coverage.py

Field	Value
Scripts	`scripts/docs/render_md_to_pdf.sh`, `scripts/docs/check_pdf_coverage.py`, `scripts/docs/print.css`, `scripts/docs/print_landscape.css`, `scripts/docs/pdf_table_cols.py`
Purpose	Render the `.pdf` twin of a Markdown doc without clipping wide tables or long tokens, and prove it by comparing the words of the Markdown with the text extracted from the PDF (constitution 11.4.65, 11.4.201)
Why	PDFs rendered from the pandoc HTML with default print rules were cut off at the right page edge (up to 29% of a doc's words missing); the file opened fine, so nothing noticed
Render	`render_md_to_pdf.sh FILE.md ...` writes `FILE.pdf` (pandoc gfm to HTML5 with `--metadata document-css=false`, because pandoc's own `<style>` (body `max-width: 36em`) outranks the print sheets in WeasyPrint; `pdf_table_cols.py` adds computed table column widths; then WeasyPrint with `print.css`). The PDF's Keywords field carries `src-sha256:<hex>` of the Markdown
Orientation rule	A doc whose Markdown has a table separator row with 5 or more columns is rendered A4 landscape (`print_landscape.css`), all others A4 portrait
Print rules	Body 9.5pt at full page width. Tables: fixed layout at 100% width, columns sized by `pdf_table_cols.py` from SEGMENTS, not characters. In every cell a zero-width space (U+200B; WeasyPrint ignores `<wbr>`, measured) is inserted after `/`, `_`, `-` and after `.` before a letter (never between digits), so a line may break only there; `file.go:118`, `2026-11-12`, `v7.67` and ids stay whole. A column's minimum is the estimated width of its longest segment (glyph classes, capped at 30 characters) plus padding; when the minimums do not fit the page the table font shrinks (7.5 to 5.5pt); only a segment longer than its column can still break mid-segment. The rest of the line is shared by content weight. Rows never split across pages, `pre` and `code` wrap
`--check`	`render_md_to_pdf.sh --check FILE.md` writes nothing and runs the coverage check on the existing PDF
Coverage check	`check_pdf_coverage.py [--root DIR] [--allow FILE] [--min-len N] [--quiet] FILE.md ...`
Method	Distinct tokens of 4 or more characters (a token containing a digit from 2 characters) of the Markdown, Unicode letters included (Cyrillic, CJK), link targets, HTML comments and fence markers removed, NFKC, must appear in `pdftotext` output on a word boundary. A line wrap inside a token is accepted only at a position the renderer may break (after `/ _ @ = & ?`, after `-` not between digits, after `.` before a letter, or inside one separator-delimited segment of 30 or more characters), NEVER between two digits, whatever the length of the enclosing word, and only when the next piece starts on a later line and not right of the previous piece's end (so neighbouring cells cannot be glued). Two reading orders (line by line and cell by cell); the cell-by-cell one is load-bearing on wide tables
Allow-list	`scripts/docs/pdf_coverage_allow.tsv` (`path`, `token`, `reason`; the reason is required). Default: no file, nothing allowed
Control	Per file, differential on both sides: a needle taken from the PDF (one that occurs once, inside one PDF word) must be found in the real word stream and NOT in a copy in which that needle is destroyed; the same on the Markdown text; a fabricated needle must not be found. A matcher that cannot tell real from destroyed text is BLIND (exit 2). A blank PDF whose Markdown has tokens is FAIL
Exit	0 no token missing; 1 tokens missing, PDF absent, empty or unreadable; 2 BLIND, usage or tool missing
Tests	`scripts/docs/tests/run_tests.sh scripts/docs/tests/test_check_pdf_coverage.py` (stdlib runner, no pytest; `run_tests.sh` alone also runs the other docs tool tests). Renderer: short ids and `file:line` citations whole in portrait and landscape, capital-heavy ids, glyph widths not flat, zero-width-space opportunities, reproducible render across clock and working directory, orientation rule. Gate: vertical split of a digit run or segment rejected (also inside a long enclosing word), wrap geometry (same line, right of end), separator-only wraps, second reading order load-bearing, needle controls differential, blank second PDF, Unicode, forged hash, untracked docs. Independent oracle `scripts/docs/tests/oracle_citations.py` (shares no code with the gate): no digit run or citation split. Mutation runs killed: see the commit message

Reproducible: the render sets `SOURCE_DATE_EPOCH` and a PDF identifier derived from the Markdown, so two renders are byte-identical (checked on the shipped PDFs). The twin gate also covers untracked new docs under `docs/`.

Limits: a doc with no tokens of 4 or more characters (or digits) reports OK with nothing to compare. Glyph widths are class-based estimates, not measured font metrics. A segment longer than 30 characters, or a table whose segment minimums do not fit even at 5.5pt, can still break mid-segment (the text stays present; the gate allows it only inside such a segment and never inside a digit run). Citations may wrap AT separators (`playlist_` / `handler.go:95`), never inside the line number. This proves the words are present and wraps sit at allowed places, not that the page looks right. The `src-sha256` Keywords value is metadata and can be forged; the twin gate's coverage run catches a forged old-content PDF only when the new Markdown adds tokens the old PDF lacks. Pasting text out of a PDF can carry zero-width spaces inside table cells.

check_twin_presence.py

Field	Value
Script	`scripts/docs/check_twin_presence.py`

Field	Value
Purpose	Gate: every Markdown doc added or modified under `docs/` since a ref has current `.html` and `.pdf` twins (constitution 11.4.65)
Usage	`check_twin_presence.py [--root DIR] [--since REF] [--no-html-drift] [--quiet]`; `--since` defaults to the newest tag
Scope	`.md` files added or modified (`git diff --diff-filter=AM REF..HEAD`, plus working-tree changes) under `docs/`; a modified doc is in scope when it already has a twin of either kind, an added doc always is
Checks	`.html` non-empty and equal to `render_md_to_html.sh --check` output; `.pdf` non-empty, `%PDF`, readable (`pdfinfo`), its Keywords `src-sha256` equals the sha256 of the current Markdown (stale PDF fails), and `check_pdf_coverage.py` reports 0 missing (`--no-pdf-coverage`, `--no-html-drift` skip those)
Control	A doc with both twins must read complete and one missing its pdf incomplete; an empty scope is BLIND
Exit	0 all present, 1 one or more `MISSING` rows, 2 BLIND or usage
Fix	`render_md_to_html.sh FILE.md` and `render_md_to_pdf.sh FILE.md`, never hand-edit generated twins